

Real Time early Multi Stage Attack Detection

Yuvraj Sanjayrao Takey
Centre for Development of
Advanced Computing (C-DAC)
Hyderabad, India
Email : yuvraj@cdac.in

Sai Gopal Tatikayala
Centre for Development of
Advanced Computing (C-DAC)
Hyderabad, India
Email : saigopalt@cdac.in

Satyanadha Sarma Samavedam
Computer Emergency Response Team - India
Cert-In, India
Email : sarma@cert-in.org.in

Lakshmi Eswari P R
Centre for Development of
Advanced Computing (C-DAC)
Hyderabad, India
Email : prleswari@cdac.in

Mahesh Uttam Patil
Centre for Development of
Advanced Computing (C-DAC)
Hyderabad, India
Email : maheshp@cdac.in

Abstract—In recent times, attackers are continuously developing advanced techniques for evading security, stealing personal financial data, Intellectual Property (IP) and sensitive information. These attacks often employ multiple attack vectors for gaining initial access to the systems. Analysts are often challenged to identify malware objective, initial attack vectors, attack propagation, evading techniques, protective mechanisms and unseen techniques. Most of these attacks are frequently referred to as Multi stage attacks and pose a grave threat to organizations, individuals and the government. Early multistage attack detection is a crucial measure to counter malware and deactivate it. Most traditional security solutions use signature-based detection, which frequently fails to thwart zero-day attacks. Manual analysis of these samples requires enormous effort for effectively counter exponential growth of malware samples. In this paper, we present a novel approach leveraging Machine Learning and MITRE Adversary Tactic Technique and Common knowledge (ATT&CK) framework for early multistage attack detection in real time. Firstly, we have developed a run-time engine that receives notification while malicious executable is downloaded via browser or a launch of a new process in the system. Upon notification, the engine extracts the features from static executable for learning if the executable is malicious. Secondly, we use the MITRE ATT&CK framework, evolved based on the real-world observations of the cyber attacks, that best describes the multistage attack with respect to the adversary Tactics, Techniques and Procedure (TTP) for detecting the malicious executable as well as predict the stages that the malware executes during the attack. Lastly, we propose a real-time system that combines both these techniques for early multistage attack detection. The proposed model has been tested on 6000 unpacked malware samples and it achieves 98 % accuracy. The other major contribution in this paper is identifying the Windows API calls for each of the adversary techniques based on the MITRE ATT&CK

Index Terms—Windows API, machine learning, malware detection, static analysis, API calls, Portable Executable (PE), Malicious code, Tactics Techniques Procedure (TTP), Multistage attack, MITRE, Static Analysis

I. INTRODUCTION

With the continuously evolving cyber threat landscape, organizations are constantly tracking the internal network and external access points to defend against the attacks. A security breach would compromise the client's data, damage

the company's reputation and risk employee's data, Intellectual property, and physical data loss. It is challenging to provide assurance for cyber-attacks because of the accessibility of information across an organization's global connectivity, information accessibility for outside users and emerging new technologies. The rise of the targeted attacks continues to pose a serious threat to the organization as they continue to evolve and refine their tools and tactics. These targeted attacks are likely performed by a nation-state or state-sponsored groups, commonly referred to as an Advanced Threat Group (APT). APT targets government, defense, financial, legal, industrial, manufacturing, Healthcare, Banking and many more to steal, spy, disrupt and cause financial damage. APT attacks gain an initial foothold through multiple attack vectors such as spear phishing, watering hole, social engineering, and exploiting Internet-facing application vulnerabilities. Once an attacker gains an initial foothold, malware spreads across the internal network, moving from one system to another system slowly to evade detection and finally accomplish the attacker's objective to steal, spy, disrupt and hide their tracks. These APT attacks often fall under multistage attack

During the multistage attacks, attackers drop malign executable on the target system and attempt to spread across the organizations to achieve the final goal. These executable evade detection as most of the antivirus follows signature-based detection. The attacker repeatedly either employs obfuscation techniques to generate polymorphic malware or discover new tactics and techniques to evade detection. Detection of malign executable hinders the detection of multistage attacks. Many approaches have been developed to detect malicious executable, but most of the solutions identify the executable based on the characteristics of the malware families like viruses, worms, Trojans, bots etc [1]–[3]. There has been a steady rise in the solutions leveraging Machine Learning (ML) and deep learning for detecting malicious executable [4], [5]. ML is a method of data analysis that learns itself without explicit programming and facilitates in discovering the hidden patterns and makes decisions with minimal human

intervention. Most of the researchers focused on static and dynamic analysis for detecting malicious executable [1], [6]. In static analysis, PE structure is examined for anomalies without actually executing it [7]–[9]. PE attributes, API calls, API sequence, opcode and byte sequence, Printable Strings (PSI), Function length Frequency are the most commonly used features for detecting the malicious executable [2], [10]–[12]. However the API and opcode sequences are challenging if the executable is obfuscated. Though there has been a lot of research on detecting the malicious executable in the past, it mainly focuses on the limited set of Windows API calls and none of them are comprehensive [13]–[15]. In this paper, we focus on static analysis as it yields rich information without executing the malware

In this paper, we propose a novel framework for early multistage attack detection in real time. We have investigated the MITRE ATT&CK framework for understanding the adversary Tactics, Techniques, and Procedures (TTP) which are based on real observation of the cyber attacks [16], [17]. TTP plays a significant role for the analyst to understand the enemy or threat actor, identify the blind spots, and facilitate in implementing corrective measures in advance. In this study, extensive research has been carried out to understand Adversary techniques for identifying potential Windows API calls. The outcome of the research facilitates in identifying the potential API for each of the TTP. For instance, input capture is one technique popularly used by adversaries for obtaining credentials for accounts. Most frequently used API calls for input capture are SetWindowsHookEx, GetKeyState, GetWindowText, ProcessKeyboardInputWorker, GetKeyboardState, RegisterHotKey, WriteFile. Similarly, we have identified around 240 plus potential windows API calls from 300+ TTP. The knowledge gained in MITRE serves two purposes

- Predicting the stages that malware will execute during the attack
- Building a malware dataset with 240 API calls along with binary file metadata

Various machine learning models are built using malware dataset to determine the malicious executables. A run-time engine is developed to deploy the solution in real time. To summarize our contribution for early multi stage attack detection, our proposed system would perform static analysis on the Portable Executable (PE) binary, predict the malicious binaries by leveraging machine learning and map these binaries with the potential techniques employed by the adversary.

The rest of the paper is organized as follows. In Section II, discusses current research related to multistage attack detection. Section III presents a brief overview of the MITRE ATT&CK and readers familiar with it can skip this section. Section IV describes proposed methodology in early detection of multistage attack. Section V shows the results of our experimentation. We conclude our study in section VI.

II. RELATED WORK

In this section, we discuss related work carried out in Multi-Stage attack detection. Emmanouil Vasilomanolakis et

al. [18] proposed multi-stage attack detection using honeypots for Industrial Control System (ICS). The author proposes novel architecture detects multi-stage attacks and automatically generates signatures. This signature would be fed to the Intrusion detection system for detecting and blocking future attacks.

The Authors [19] propose a framework which leverages Intrusion Kill Chain and defense patterns for detecting multi-stage attacks. The framework is evaluated on a pdf and demonstrated the multi-stage attack. Though it includes the CKC concept in the design, it didn't consider the adversary techniques as it plays an critical role in detecting the multi-stage attack

Sunu Mathew et al. [20] propose real time Multi-stage attack that categorizes classification of intrusion detection sensor alerts based on their role as part of goal oriented multi-stage attacks. The authors demonstrated multistage attack incorporating awareness indicators as well as attack stage progression. However, this paper focus on the network traffic for detecting multi-stage and didn't correlate the host events across end system for detecting multi-stage attacks

Francisco et al. [21] propose multi-stage attack detection using contextual information. The author proposes a new approach of identifying the Point of Entry (PoE) to a target system which an attacker would be using for exploiting the system. The author claims that the proposed solution detected 58% of attacks in real-time. However, this system didn't correlate the network and host events. Instead focus mainly on PoE of the target system

There is a lot of research on classification of malware based on the API [10]–[15]. But none on identifying and predicting the adversary techniques using Window's Native API. In this paper, we focus on predicting whether the binary is malicious or not and map the binary to the adversary technique based on the windows native API. Compared to other researcher's work, we have conducted a broad investigation/analysis to satisfy the gaps and concentrated on the main aspect for detecting the multi-stage attack.

III. BACKGROUND

Lockheed Martin proposed Cyber Kill Chain (CKC) framework for identification and prevention of cyber attacks. CKC is an adversary model that describes the sequence of stages in the Multistage attack. Each stage accommodates a specific goal and the adversary needs to execute a chain of stages for achieving the attackers final objective. CKC framework is a powerful concept to detect multistage attack and defend against the attack by breaking the chain. CKC proposes seven distinct stages namely Reconnaissance, Weaponization, Delivery, Exploitation, Installation, Command & Control (C&C) and Actions on objectives. CKC provides an intelligence framework in understanding the multistage attack, but fails to provide the information about adversaries Tactics and Techniques. However, CKC is a starting point for understanding the multistage attack. In contrast, MITRE ATT&CK focus on the adversary Tactics, Techniques and procedure (TTP) derived from the real time attacks. It defines 12 Tactics which have

TABLE I
MITRE TACTIC

MITRE Tactic	Objective	Adversary Techniques
Initial Access	Adversary employ multiple attack vector to gain initial footholds within a network	Drive-by-Download, Spear-phishing
Execution	Different mechanism through which adversary run malicious code on local or remote system	Control Panel items, Execution through API
Persistence	Techniques for keeping access to system across reboots	Logon scripts, New service
Privilege escalation	Techniques for gaining high level permissions on a system or network	Bypass User Access Control, Hooking
Defensive Evasion	Techniques to avoid detection of security solutions	Access Token Manipulation, code signing
Credential access	Techniques for stealing username and password details	Hooking, Input Capture
Discovery	Techniques for gaining knowledge of the host and network environment	Account Discovery, Network Service Scanning
Lateral Movement	Techniques to enter and control remote system in the network	Pass the Hash, Remote Desktop Protocol
Collection	Techniques for gathering information of target system	Audio capture, screen capture
Command & Control	Techniques to communicate with victim system and issue command	Data Encoding, Data obfuscation
Impact	Technique to disrupt availability or compromise integrity	Data Destruction, Account access Removal

earlier been referred to as stages in CKC. Many adversary techniques are discovered based on the real observation of attack data for each tactic and the adversary chose one technique out of many to accomplish the specific tactic. Along with adversary technique, detailed procedure of the technique which includes registry, configuration, startup services, file creation/modification/deletion and network changes. MITRE ATT&CK is a collection of 300 plus adversary TTP's . Table-I shows MITRE Tactics and their corresponding adversary techniques. For simplicity, only two adversary techniques are listed under each tactic

IV. PROPOSED METHODOLOGY

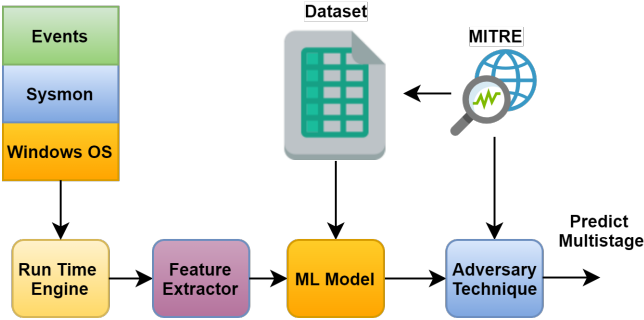


Fig. 1. Overall Architecture

Figure 1 shows the overall architecture for early detection of multistage attack. Major component present are

- 1) Run Time Engine
- 2) Machine Learning components
 - Feature Extractor
 - ML Engine
 - Dataset
- 3) MITRE Adversary Technique

Run-time engine reads the process events from Windows Operating System (OS) to determine whether the executable is malicious or not. It is a core component which is invoked either a process is created or a file download from the web browser. Windows OS provides event-logging service for applications and Operating System (OS) to record software and hardware events. All system and applications registry with event logging service to notify the changes

in the OS. All the events are stored under a specific category in the C:/Windows/system32/config. Typically, these events are used for diagnosing and troubleshooting the system. However, events generated are huge and event raw data doesn't provide much information related to malicious activities unless correlated with the other events. On the other hand, Microsoft System Monitor (Sysmon) offers more concrete evidence of malicious events that facilitate the discovery of the malware. Runtime engine register with Sysmon for collecting detailed information about information about process creation, network connections and modification to file creation time . Sysmon is an application monitor that writes critical events to the Applications and Services Logs/Microsoft/Windows/Sysmon/Operational.

Sysmon allows 23 critical events to be notified for the registered application. Out of 23 events, we have opted for two events

- Event ID 1: Process creation
 - Event provides information about the newly created process
- Event ID 15: FileCreateStreamHash
 - Most malware drop executables or configuration settings via browser download. This event facilitates capturing these malicious events. Even this feature helps in identifying the spear phishing attack

Upon receiving the process creation event or Web browser file download, the run-time engine reads the executable file and passes it to the next stage. The next stage extracts the features from the static binary and pass it to the machine learning for detection of malicious executable.

Before proceeding to feature extraction, we first discuss on the dataset to understand how the distinguished features are identified for detecting multistage attack. As mentioned earlier, MITRE TTP are employed to gain insight of the adversary technique. Each adversary technique has been mapped to the Data Sources (DS) and these DS facilitates in detecting the multistage attack. MITRE has defined 52 DS for identifying the multistage attacks namely API Monitoring, System calls, Process use of network, Binary File Metadata, DLL Monitoring, Malware Reverse Engineering, Network Protocol Analysis, Network Intrusion Detection System, Application logs, process monitoring and many more. In this research,

TABLE II
FEATURES DERIVED FROM MITRE DATA SOURCE

Data Source	Feature Name	Feature Number
API	API calls	F1-F244
Binary File Metadata	Export Table	F245
	Import Table	F246
	Ordinal present	F247
	Architecture	F248
	ValidEntryPoint	F249
	ValidRawSize	F250
	ValidHeader	F251
	StdSection	F252
	Signed	F253
	Section count	F254
	Function count	F255
	Size of Initialized data	F256
	ispacked	F257
	isdebug	F258
DLL Monitoring	DLL count	F259

we are focusing on the DS which would be extracted from the Windows Portable Executable (PE). PE format describes the structure of executable, Dynamic Linking Library (DLL) and object files which are compatible with all versions of Windows OS. PE files encapsulate different sections which contain information for Windows loader to manage and wrap the executable code [8]. PE file header contains MS-DOS stub, File header, optional header, section header and their corresponding sections.

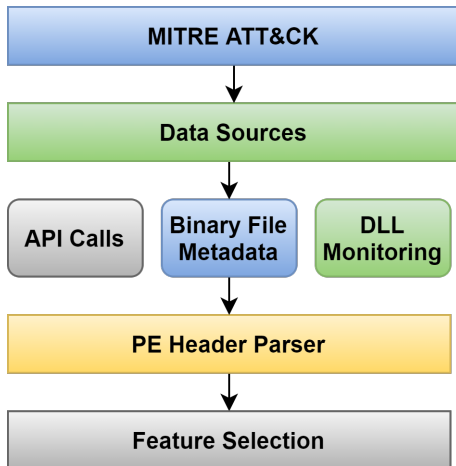


Fig. 2. Labeled Dataset

Figure 2 process of labeling the dataset. The major challenge is to identify potential features from adversary techniques and build a labeled dataset. We have examined various adversary techniques that are identified using API, Binary File Meta data and DLL monitoring DS. Based on the literature survey and real time observations of the attack, potential features are identified [1], [10], [11], [13]–[16], [22]. Table-II shows the potential features for each DS, a total of 245 potential features are identified.

Feature extraction is a critical component of machine learning which reads the PE file passed from the runtime engine and extracts the features. To extract features from PE file,

python along with a pefile library is used. Model would take the features as the input and label the executable as malign or benign. If the executable is malicious, it predicts the stages that the malware executes during the attack. After examining the MITRE TTP, we have identified the API calls for various adversary techniques. Table-III shows the API that would be executed for each adversary technique

V. EXPERIMENTAL RESULTS

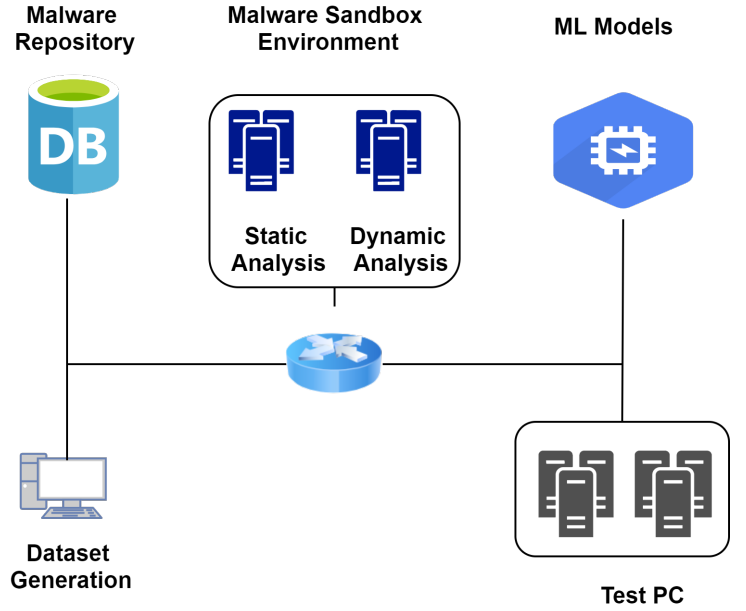


Fig. 3. Lab Setup and Deployment

In this discussion, we discuss the testbed for evaluating the Machine learning model. Figure 3 shows the experimental setup for training the machine learning models and deployment of the model in real time. As explained earlier, the dataset generation is carried out by identifying the potential features from MITRE adversary technique and literature survey. Malware samples are collected from the publicly available sites, Cert-In, India and Honeypot and are kept in the malware repository. For each malware sample features are extracted from windows executable and labeled as malign. Similarly, we have collected the all executable present in the C folder from a freshly installed Windows 10 System, extracted features and labeled as benign. Labeled dataset contains 7699 entries out of which 4076 are labeled as malign and 3623 are labeled as benign. Naive Bayes, Decision Tree, KNN, SVM and Random Forest machine learning models are trained on the labeled dataset . After applying Principal Component Analysis (PCA) for reducing the feature size, feature size is 160 as compared to 257. The accuracy achieved is 98.18 %. Fig 4 shows the predicted adversary techniques executed by the malware and predicted stages are highlighted

Execution	Persistence	Privilege_Escalation	Defense_Evasion	Credential_Access	Discovery	Lateral_Movement	Collection Impact	Extras
Control Panel Items	Account Manipulation	Access Token Manipulation	Access Token Manipulation	Input Capture	Account Discovery	Distributed Component Object Model	Audio Capture	Service Stop Anti-Debugging
Dynamic Data Exchange	BITS Jobs	Extra Window Memory Injection	BITS Jobs	Account Manipulation	Application Window Discovery	-	-	Anti-VM
Execution through API	Bootkit	Hooking	Component Firmware	Credential Dumping	Browser Bookmark Discovery	-	-	Packers
Execution through Module Load	Component Firmware	Port Monitors	Control Panel Items	Hooking	Domain Trust Discovery	-	-	-
LSASS Driver	Hooking	Process Injection	DCShadow	Two-Factor Authentication Interception	Permission Groups Discovery	-	-	-
-	LSASS Driver	SID-History Injection	Disabling Security Tools	-	System Time Discovery	-	-	-
-	Port Monitors	-	Extra Window Memory Injection	-	-	-	-	-
-	SIP and Trust Provider Hijacking	-	File System Logical Offsets	-	-	-	-	-
-	System Firmware	-	Indicator Removal on Host	-	-	-	-	-
-	Time Providers	-	NTFS File Attributes	-	-	-	-	-
-	-	-	Process Doppelganging	-	-	-	-	-
-	-	-	Process Hollowing	-	-	-	-	-
-	-	-	Process Injection	-	-	-	-	-
-	-	-	SIP and Trust Provider Hijacking	-	-	-	-	-

Fig. 4. Mapping Binary to MITRE phases

TABLE III: MITRE Adversary Techniques and their corresponding API

MITRE Tactic	Adversary Technique	API calls
Execution	Control Panel Items	Control_RunDLL, Control_RunDLLAsUser
	Dynamic Data Exchange	DdeSetQualityOfService, FreeDDEIParam, ImpersonateDdeClientWindow, PackDDEIParam, ReuseDDEIParam, UnpackDDEIParam
	Execution through API	CreateProcess, CreateProcessAsUser, CreateProcessInternal, CreateProcessWithLogon, LoadLibrary, LoadLibraryEx, LoadModule, LoadPackagedLibrary, WinExec, ShellExecute, ShellExecuteEx
	Execution through Module Load	LoadLibrary, CreateProcess
Persistence	BITS Jobs	CoInitializeEx, CoCreateInstance, CreateJob, GetJob, AddFile, SetNotifyFlags, SetCredentials, Resume, GetProxySettings, SetPriority, TakeOwnership
	Hooking	SetWindowsHookEx, CallNextHookEx, CreateToolhelp32Snapshot, Process32First, Process32Next, Thread32First, Thread32Next, LoadLibrary, GetProcAddress
	Port Monitors	AddMonitor, WinINet, EnumMonitors, EnumPrinters, GetPrinterDriver, GetCorePrinterDrivers, GetPrinterDriverPackagePath, GetPrintProcessorDirectory, InstallPrinterDriverFromPackage, UploadPrinterDriverPackage
	SIP and Trust Provider Hijacking	CryptSIPDllGetSignedDataMsg, CryptSIPDllVerifyIndirectData, CryptSIPLoad, CryptSIPRetrieveSubjectGuid, CryptSIPAddProvider
	Time Providers	GetSystemTimePreciseAsFileTime, AlertSamplesAvailFunc, GetTimeSysInfoFunc, LogTimeProvEventFunc, TimeProvOpen
Privilege Escalation	Access Token Manipulation	DuplicateToken, ImpersonateLoggedOnUser, SetThreadToken, DuplicateToken, CreateProcessWithToken, LogonUser
	Extra Window Memory Injection	SetWindowLong, GetWindowLong, SendNotifyMessage
	Hooking	SetWindowsHookEx, CallNextHookEx, CreateToolhelp32Snapshot, Process32First, Process32Next, Thread32First, Thread32Next, LoadLibrary, GetProcAddress

	Port Monitors	AddMonitor, WinINet, EnumMonitors, EnumPrinters, GetPrinterDriver, GetCorePrinterDrivers, GetPrinterDriverPackagePath, GetPrintProcessorDirectory, InstallPrinterDriverFromPackage, UploadPrinterDriverPackage
	SID-History Injection	DsAddSidHistory
	Process Injection	WriteProcessMemory, CreateRemoteThread, OpenProcess, VirtualAllocEx, CreateToolhelp32Snapshot, Process32First, Process32Next, Thread32First, Thread32Next, GetThreadContext, CreateProcess, NtUnmapViewOfSection, ZwUnmapViewOfSection, SetThreadContext, ResumeThread, RegCreateKeyEx, RegSetValueEx, CreateProcessAsUser, CreateProcessWithLogon, CreateProcessWithToken, WinExec, QueueUserAPC, SetWindowLong, GetWindowLong, SendNotifyMessage, CreateWindowEx, VirtualProtect, GetStartupInfo, HeapCreate, IsValidCodePage
	Parent PID Spoofing	CreateProcess, UpdateProcThreadAttribute
Defense Evasion	Access Token Manipulation	DuplicateToken, ImpersonateLoggedOnUser, SetThreadToken, DuplicateToken, CreateProcessWithToken, LogonUser
	BITS Jobs	CoInitializeEx, CoCreateInstance, CreateJob, GetJob, AddFile, SetNotifyFlags, SetCredentials, Resume, GetProxySettings, SetPriority, TakeOwnership
	Control Panel Items	Control_RunDLL, Control_RunDLLAsUser
	Extra Window Memory Injection	SetWindowLong, GetWindowLong, SendNotifyMessage
	NTFS File Attributes	ZwSetEaFile, ZwQueryEaFile
	Process Doppelganging	CreateTranscation, CommitTransaction, CreateFileTransacted, CreateDirectoryTransacted, RollbackTransaction, NtCreateProcessEx, NtCreateThreadEx, WriteProcessMemory
	Process Hollowing	WriteProcessMemory, CreateRemoteThread, OpenProcess, VirtualAllocEx, CreateToolhelp32Snapshot, Process32First, Process32Next, Thread32First, Thread32Next, GetThreadContext, CreateProcess, NtUnmapViewOfSection, ZwUnmapViewOfSection, SetThreadContext, ResumeThread, RegCreateKeyEx, RegSetValueEx, CreateProcessAsUser, CreateProcessWithLogon, CreateProcessWithToken, WinExec, QueueUserAPC, SetWindowLong, GetWindowLong, SendNotifyMessage, CreateWindowEx, VirtualProtect
	SIP and Trust Provider Hijacking	CryptSIPDllGetSignedDataMsg, CryptSIPDllVerifyIndirectData, CryptSIPLoad, CryptSIPRetrieveSubjectGuid, CryptSIPAddProvider
Credential Access	Parent PID Spoofing	CreateProcess, UpdateProcThreadAttribute
	Input Capture	SetWindowsHook, SetWindowsHookEx, GetKeyState, GetAsyncKeyState, GetWindowText, GetKeyboardState, ProcessKeyboardInputWorker, ProcessKeyboardInput, GetCursorPos
	Hooking	SetWindowsHookEx, CallNextHookEx, CreateToolhelp32Snapshot, Process32First, Process32Next, Thread32First, Thread32Next, LoadLibrary, GetProcAddress

	Credential Dumping	OpenSamUser, CloseSamUser, InitSecurityInterface, QuerySecurityPackageInfo, AcquireCredentialsHandle, ExportSecurityContext, QueryCredentialsAttributes, ImportSecurityContext, AcceptSecurityContext, CompleteAuthToken, ImpersonateSecurityContext, InitializeSecurityContext, QueryContextAttributes, DecryptMessage, EncryptMessage, LsaApCallPackage, LsaApInitializePackage, LsaApLogonUser, SpAcceptCredentials, SpAcquireCredentialsHandle, SpAcceptLsaModeContext, SpAddCredentials, SpGetCredentials, SpInitLsaModeContext, SPInitialize, LsaCallAuthenticationPackage, LsaConnectUntrusted, LsaEnumerateLogonSessions, LsaLogonUser, LsaRegisterLogonProcess, AddCredential, AllocateLsaHeap, DeleteCredential, CredUIConfirmCredentials, CredUIReadSSOCred, CredEnumerate, CredFindBestCredential, CredRead, CredRename, CredWrite, CredWriteDomainCredentials, NPLogonNotify, CredGetTargetInfo, SaslAcceptSecurityContext, SaslEnumerateProfiles, SaslGetContextOption, SaslInitializeSecurityContext, SaslSetContextOption, ChangeAccountPassword, LsaManageSidNameMapping, LsaOpenPolicy, SpLsaModeInitialize, AddSecurityPackage, DeleteSecurityPackage
Discovery	Account Discovery	CoInitializeEx, CoInitializeSecurity, CoSetProxyBlanket, CoCreateInstance, ConnectServer
	Domain Trust Discovery	DSEnumerateDomainTrusts, LsaQueryTrustedDomainInfo, LsaCreateTrustedDomainEx, LsaDeleteTrustedDomain, LsaSetTrustedDomainInformation
	System Time Discovery	GetSystemTime, GetSystemTimeAsFileTime, GetSystemTimeAdjustment, SetSystemTimeAdjustment, GetTickCount
Lateral Movement	Distributed Component Object Model	CreateObject, CoCreateInstanceEx, CoInitializeEx, CreateSession, CoGetMalloc, CoInstall, CoLoadLibrary, CoQueryClientBlanket, Initialize, RpcBindingCreate, RpcBindingBind, StringFromCLSID, CoRegisterClassObject, RpcServerListen, listen, accept, connect, socket, send, recv, recvfrom, sendto, bind
Collection	Audio Capture	ActivateAudioInterfaceAsync, waveInOpen
	Clipboard Data	GetClipboardSequenceNumber, SetClipboardViewer, GetClipboardViewer, ChangeClipboardChain, OpenClipboard, CloseClipboard, EmptyClipboard, RegisterClipboardFormat, GetClipboardData, SetClipboardData
	Input Capture	SetWindowsHook, SetWindowsHookEx, GetKeyState, GetAsyncKeyState, GetWindowText, GetKeyboardState, ProcessKeyboardInputWorker, ProcessKeyboardInput, GetCursorPos
	Screen Capture	SetWindowsHookEx, BitBlt, WriteFile, SelectObject, CreateCompatibleBitmap, CreateBitmap, CreateCompatibleDC, GetDeviceCaps, GetDC, GetWindowDC
Impact	Service Stop	ChangeServiceConfig, ControlService, CloseServiceHandle, CreateService, DeleteService, OpenSCManager, OpenService, QueryServiceConfig, EnumServicesStatusEx, StartService, QueryServiceStatusEx, SetServiceObjectSecurity

VI. CONCLUSION

In this paper, we present a novel approach leveraging Machine Learning and MITRE Adversary Tactic Technique and Common knowledge (ATT&CK) framework for early multistage attack detection in real time. We have evolved a real time solution that monitors the process creation, dropped binaries through the browser and determines the executable is malicious or not based on PE structure. To identify potential

features, a lot of literature has been explored, experimented along with the understanding of Adversary TTP. In addition, we have investigated the Adversary techniques and identified the potential API for each of the possible adversary technique. Eventually, the proposed solution will predict the stages that the malware executes during the attack based on the API identified using MITRE framework. The results demonstrate that leveraging Machine learning and MITRE is a powerful

method for detecting and predicting Multi-Stage attack

In future work, we intend to develop more sophisticated techniques for identifying the multistage attack. The planned solution will incorporate the real time events generated from the remaining Data Sources and map with the MITRE adversary techniques. Thus, early multistage detection along with the planned work would facilitate in detection of advanced Multi-Stage attack. This could be valuable for discovering new adversary techniques employed and also to understand the complete end to end activities carried out during the attack.

REFERENCES

- [1] N. Kumar, S. Mukhopadhyay, M. Gupta, A. Handa, and S. K. Shukla, "Malware classification using early stage behavioral analysis," in *2019 14th Asia Joint Conference on Information Security (AsiaJCIS)*. IEEE, 2019, pp. 16–23.
- [2] R. Islam, R. Tian, L. Batten, and S. Versteeg, "Classification of malware based on string and function feature selection," in *2010 Second Cybercrime and Trustworthy Computing Workshop*. IEEE, 2010, pp. 9–17.
- [3] R. Vyas, X. Luo, N. McFarland, and C. Justice, "Investigation of malicious portable executable file detection on the network using supervised learning techniques," in *2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*. IEEE, 2017, pp. 941–946.
- [4] O. E. David and N. S. Netanyahu, "Deepsign: Deep learning for automatic malware signature generation and classification," in *2015 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2015, pp. 1–8.
- [5] H. Jiang, T. Turki, and J. T. Wang, "Dlgraph: Malware detection using deep learning and graph embedding," in *2018 17th IEEE international conference on machine learning and applications (ICMLA)*. IEEE, 2018, pp. 1029–1033.
- [6] D. Kim, S. Woo, D. Lee, and T. Chung, "Static detection of malware and benign executable using machine learning algorithm," in *INTERNET 2016: The Eighth International Conference on Evolving Internet*, 2016, pp. 14–19.
- [7] T.-Y. Wang, C.-H. Wu, and C.-C. Hsieh, "Detecting unknown malicious executables using portable executable headers," in *2009 Fifth International Joint Conference on INC, IMS and IDC*. IEEE, 2009, pp. 278–284.
- [8] M. Z. Shafiq, S. M. Tabish, F. Mirza, and M. Farooq, "Pe-miner: Mining structural information to detect malicious executables in realtime," in *Recent Advances in Intrusion Detection*, E. Kirda, S. Jha, and D. Balzarotti, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 121–141.
- [9] E. Raff, J. Sylvester, and C. Nicholas, "Learning the pe header, malware detection with minimal domain knowledge," in *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, 2017, pp. 121–132.
- [10] G. G. Sundarkumar, V. Ravi, I. Nwogu, and V. Govindaraju, "Malware detection via api calls, topic models and machine learning," in *2015 IEEE International Conference on Automation Science and Engineering (CASE)*. IEEE, 2015, pp. 1212–1217.
- [11] M. Belaoued and S. Mazouzi, "Statistical study of imported apis by pe type malware," in *2014 International Conference on Advanced Networking Distributed Systems and Applications*. IEEE, 2014, pp. 82–86.
- [12] M. Wang, C. Zhang, and J. Yu, "Native api based windows anomaly intrusion detection method using svm," in *IEEE International Conference on Sensor Networks, Ubiquitous, and Trustworthy Computing (SUTC'06)*, vol. 1. IEEE, 2006, pp. 6–pp.
- [13] J. Y.-C. Cheng, T.-S. Tsai, and C.-S. Yang, "An information retrieval approach for malware classification based on windows api calls," in *2013 International conference on machine learning and cybernetics*, vol. 4. IEEE, 2013, pp. 1678–1683.
- [14] A. Pektaş and T. Acarman, "Malware classification based on api calls and behaviour analysis," *IET Information Security*, vol. 12, no. 2, pp. 107–117, 2018.
- [15] F. Ahmed, H. Hameed, M. Z. Shafiq, and M. Farooq, "Using spatio-temporal information in api calls with machine learning algorithms for malware detection," in *Proceedings of the 2nd ACM workshop on Security and artificial intelligence*, 2009, pp. 55–62.
- [16] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "Mitre att&ck: Design and philosophy," *Technical report*, 2018.
- [17] "Mitre attck," <https://attack.mitre.org/>.
- [18] E. Vasilomanolakis, S. Srinivasa, C. G. Cordero, and M. Mühlhäuser, "Multi-stage attack detection and signature generation with ics honeypots," in *NOMS 2016-2016 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2016, pp. 1227–1232.
- [19] P. Bhatt, E. T. Yano, and P. Gustavsson, "Towards a framework to detect multi-stage advanced persistent threats attacks," in *2014 IEEE 8th international symposium on service oriented system engineering*. IEEE, 2014, pp. 390–395.
- [20] S. Mathew, D. Britt, R. Giomundo, S. Upadhyaya, M. Sudit, and A. Stotz, "Real-time multistage attack awareness through enhanced intrusion alert clustering," in *MILCOM 2005-2005 IEEE Military Communications Conference*. IEEE, 2005, pp. 1801–1806.
- [21] F. J. Aparicio-Navarro, K. G. Kyriakopoulos, I. Ghafir, S. Lambotharan, and J. A. Chambers, "Multi-stage attack detection using contextual information," in *MILCOM 2018-2018 IEEE Military Communications Conference (MILCOM)*. IEEE, 2018, pp. 1–9.
- [22] K. Raman *et al.*, "Selecting features to classify malware," *InfoSec Southwest*, vol. 2012, 2012.